Finally, the `update_all` class method allows us to specify the `set` and `where` clauses of the SQL `update` statement. For example, the following increases the prices of all products with *Java* in their title by 10 percent:

```
result = Product.update_all("price = 1.1*price", "title like '%Java%'")
```

The return value of `update_all` depends on the database adapter; most (but not Oracle) return the number of rows that were changed in the database.

save, save!, create, and create!

It turns out that there are two versions of the `save` and `create` methods. The variants differ in the way they report errors.

- `save` returns `true` if the record was saved; it returns `nil` otherwise.
- `save!` returns `true` if the save succeeded; it raises an exception otherwise.
- `create` returns the Active Record object regardless of whether it was successfully saved. You'll need to check the object for validation errors if you want to determine whether the data was written.
- `create!` returns the Active Record object on success; it raises an exception otherwise.

Let's look at this in a bit more detail.

Plain old `save` returns `true` if the model object is valid and can be saved:

```
if order.save
  # all OK
else
  # validation failed
end
```

It's up to us to check on each call to `save` to see that it did what we expected. The reason Active Record is so lenient is that it assumes `save` is called in the